Obliczenia Naukowe - Lista 1

Jakub Retmańczyk

25 października 2026

1 Zadanie 1

Wyznaczyłem iteracyjnie epsilon maszynowy (macheps), najmniejszą dodatnią liczbę maszynową (η) oraz największą liczbę skończoną (MAX) dla typów `Float16`, `Float32` i `Float64`. Wyniki porównałem z funkcjami wbudowanymi Julii oraz wartościami z `float.h`.

Tabela 1: Porównanie parametrów arytmetyki wyznaczonych iteracyjnie ze stałymi bibliotecznymi.

Parametr	**Float16**	**Float32**	**Float64**
macheps (obliczone)	$9.765625 \cdot 10^{-4}$	$1.19209290 \cdot 10^{-7}$	$2.220446049250313 \cdot 10^{-16}$
Wartość teoretyczna	2^{-10}	2^{-23}	2^{-52}
`eps(T)`	$9.765625 \cdot 10^{-4}$	$1.19209290 \cdot 10^{-7}$	$2.220446049250313 \cdot 10^{-16}$
`float.h` (`*_EPSILON`)	-	$1.19209290 \cdot 10^{-7}$	$2.220446049250313 \cdot 10^{-16}$
η (obliczone)	$5.960464 \cdot 10^{-8}$	$1.40129846 \cdot 10^{-45}$	$4.940656458412465 \cdot 10^{-324}$
Wartość teoretyczna	2^{-24}	2^{-149}	2^{-1074}
`nextfloat(T(0.0))`	$5.960464 \cdot 10^{-8}$	$1.40129846 \cdot 10^{-45}$	$4.940656458412465 \cdot 10^{-324}$
`floatmin(T)`	$6.103516 \cdot 10^{-5}$	$1.17549435 \cdot 10^{-38}$	$2.225073858507201 \cdot 10^{-308}$
Wartość teoretyczna	2^{-14}	2^{-126}	2^{-1022}
`float.h` (`*_MIN`)	-	$1.17549435 \cdot 10^{-38}$	$2.225073858507201 \cdot 10^{-308}$
MAX (obliczone)	65504.0	$3.40282347 \cdot 10^{38}$	$1.7976931348623157 \cdot 10^{308}$
Wartość teoretyczna	$(2 - 2^{-10}) \cdot 2^{15}$	$(2 - 2^{-23}) \cdot 2^{127}$	$(2 - 2^{-52}) \cdot 2^{1023}$
`floatmax(T)`	65504.0	$3.40282347 \cdot 10^{38}$	$1.7976931348623157 \cdot 10^{308}$
`float.h` (`*_MAX`)	-	$3.40282347 \cdot 10^{38}$	$1.7976931348623157 \cdot 10^{308}$

Interpretacja

- Precyzja arytmetyki to połowa epsilona maszynowego: $\epsilon = \frac{1}{2}\text{macheps} = 2^{-t-1}$.
- Wartość η to najmniejsza dodatnia liczba maszynowa większa od zera ($2^{e_{\min}-t}$, czyli 2^{-1074} w `Float64`), odpowiadająca `nextfloat(zero(T))`. Funkcja `floatmin` zwraca natomiast najmniejszą liczbę o pełnej precyzji mantysy ($2^{e_{\min}} = 2^{-1022}$). Zachodzi relacja $\eta = \text{macheps} \cdot \text{floatmin}$.
- Wyznaczone iteracyjnie parametry macheps, η oraz MAX są ściśle zgodne ze stałymi bibliotecznymi Julii oraz `float.h`.

2 Zadanie 2

Zbadałem, czy liczby `Float64` są równomiernie rozmieszczone w przedziale $[1, 2]$ z krokiem $\delta = 2^{-52}$ oraz jak wygląda rozmieszczenie w $[1/2, 1]$ i $[2, 4]$.

Dla każdego przedziału sprawdziłem 10^6 kolejnych kroków `nextfloat`. Zależność $x = a + k \cdot \delta_a$ była spełniona w każdym kroku.

Tabela 2: Kroki siatki liczb maszynowych w badanych przedziałach (`Float64`).

Przedział	**Krok zmierzony δ_a**	**Krok teoretyczny**	**Zgodność (10^6 kroków)**
$[1/2, 1]$	$\approx 1.110223 \cdot 10^{-16}$	$2^{-53} = \frac{1}{2}\delta$	Tak
$[1, 2]$	$\approx 2.220446 \cdot 10^{-16}$	$2^{-52} = \delta$	Tak
$[2, 4]$	$\approx 4.440892 \cdot 10^{-16}$	$2^{-51} = 2\delta$	Tak

Tabela 3: Analiza bitowa wybranych punktów (`bitstring`).

Wartość	**Znak**	**Cecha (11 bitów)**	**Wykładnik e**	**Mantysa (skrót)**
0.5	0	`01111111110`	1022 (2^{-1})	`0000000000 ...0000000000`
`nextfloat(0.5)`	0	`01111111110`	1022 (2^{-1})	`0000000000 ...0000000001`
1.0	0	`01111111111`	1023 (2^{0})	`0000000000 ...0000000000`
`nextfloat(1.0)`	0	`01111111111`	1023 (2^{0})	`0000000000 ...0000000001`
$1.0 + 2\delta$	0	`01111111111`	1023 (2^{0})	`0000000000 ...0000000010`
2.0	0	`10000000000`	1024 (2^{1})	`0000000000 ...0000000000`
`nextfloat(2.0)`	0	`10000000000`	1024 (2^{1})	`0000000000 ...0000000001`

Interpretacja

- Liczby `Float64` są rozmieszczone lokalnie równomiernie w przedziałach $[2^p, 2^{p+1})$ ze stałym krokiem siatki $\delta_p = 2^{p-52}$.
- Każdą liczbę z przedziału $[2^p, 2^{p+1})$ można przedstawić jako $x = 2^p + k \cdot 2^{p-52}$ dla $k = 0, 1, \ldots, 2^{52} - 1$.
- Przekroczenie każdej kolejnej potęgi dwójki podwaja odległość między sąsiednimi liczbami maszynowymi.

3 Zadanie 3

Szukałem w arytmetyce `Float64` liczby $x \in (1, 2)$ takiej, że $\mathrm{fl}(x \cdot \mathrm{fl}(1/x)) \neq 1.0$.

Wyniki

- Zadanie 3a (dowolny przykład): już `nextfloat(1.5)` spełnia warunek:

$$x = 1.5 + 2^{-52}, \quad \mathrm{fl}(x \cdot \mathrm{fl}(1/x)) = 1.0 - 2^{-53} \neq 1.0.$$

- Zadanie 3b (najmniejsza liczba): przeszukując od 1.0 funkcją `nextfloat`, znalazłem ją po 257 736 490 krokach:

$$x_{\min} = 1.0 + 257\,736\,490 \cdot 2^{-52} \approx 1.0000000 5722899709681,$$
$$\mathrm{fl}(x_{\min} \cdot \mathrm{fl}(1/x_{\min})) = 1.0 - 2^{-53} \neq 1.0.$$

Pseudokod

Wyszukiwanie najmniejszego $x \in (1, 2)$ takiego, że $\mathrm{fl}(x \cdot \mathrm{fl}(1/x)) \neq 1$

```
1. x ← nextfloat(1.0),    k ← 1
2. while x < 2.0 do
3.     if x · (1.0/x) ≠ 1.0 then return (x, k)
4.     x ← nextfloat(x),    k ← k + 1
5. end while
```

Interpretacja

Dla $x \in (1, 2)$ błąd zaokrąglenia odwrotności spełnia $x\delta < 2^{-53}$, co uniemożliwia zaokrąglenie iloczynu w górę; jedyną możliwą niezgodnością jest zaokrąglenie w dół do $1 - 2^{-53}$. Wymagany do tego margines błędu pojawia się po raz pierwszy dopiero przy kroku $k = 257\,736\,490$.

4 Zadanie 4

Obliczyłem iloczyn skalarny wektorów:

$$x = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]^T,$$
$$y = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]^T,$$

czterema metodami w precyzji `Float32` i `Float64`. Wartość dokładna: $S_{\text{dokładna}} = -1.00657107000000 \cdot 10^{-11}$.

Tabela 4: Wyniki iloczynu skalarnego w arytmetyce `Float32` i `Float64`.

Metoda	**Float32**		**Float64**	
	Wartość	Błąd bezwzgl.	Wartość	Błąd bezwzgl.
(a) w przód	-0.4999443	0.4999443	$1.025 \cdot 10^{-10}$	$1.126 \cdot 10^{-10}$
(b) w tył	-0.4543457	0.4543457	$-1.564 \cdot 10^{-10}$	$1.464 \cdot 10^{-10}$
(c) od największych	-0.5000000	0.5000000	0.000000	$1.007 \cdot 10^{-11}$
(d) od najmniejszych	-0.5000000	0.5000000	0.000000	$1.007 \cdot 10^{-11}$

Interpretacja

- Wynik iloczynu skalarnego (10^{-11}) powstaje z odejmowania składników rzędu 10^6, powodując redukcję cyfr znaczących.
- Precyzja `Float32` (ok. 7 cyfr dziesiętnych) wyczerpuje się na samych składnikach rzędu 10^6. Komputer traci część ułamkową i nie jest w stanie wyznaczyć różnicy rzędu 10^{-11} (stąd wynik ok. -0.5 i błąd względny rzędu 10^{10}).
- W `Float64` metody (a) i (b) dają błąd względny ponad 1100%. Metody (c) i (d) sumują składniki do dokładnie przeciwnych wartości maszynowych ($S_1 + S_2 = 0.0$); błąd bezwzględny wynosi wówczas $1.007 \cdot 10^{-11}$, lecz błąd względny to dokładnie 100% (żadna cyfra znacząca nie jest poprawna).

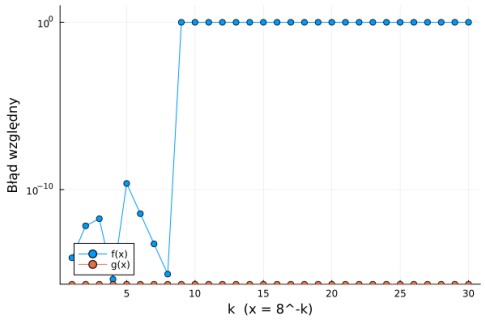
5 Zadanie 5

Zbadałem wartości dwóch algebraicznie tożsamych funkcji dla $x = 8^{-k}$ $(k = 1, \ldots, 25)$:

$$f(x) = \sqrt{x^2 + 1} - 1, \qquad g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}. \tag{1}$$

Tabela 5: Wartości $f(x)$ i $g(x)$ w arytmetyce `Float64`.

k	$x = 8^{-k}$	$f(x)$	$g(x)$
1	$1.25 \cdot 10^{-1}$	$7.782218537318641 \cdot 10^{-3}$	$7.782218537318706 \cdot 10^{-3}$
3	$1.95 \cdot 10^{-3}$	$1.907346813823096 \cdot 10^{-6}$	$1.907346813826566 \cdot 10^{-6}$
5	$3.05 \cdot 10^{-5}$	$4.656612873077393 \cdot 10^{-10}$	$4.656612871993190 \cdot 10^{-10}$
7	$4.77 \cdot 10^{-7}$	$1.136868377216160 \cdot 10^{-13}$	$1.136868377216096 \cdot 10^{-13}$
8	$5.96 \cdot 10^{-8}$	$1.776356839400250 \cdot 10^{-15}$	$1.776356839400249 \cdot 10^{-15}$
9	$\mathbf{7.45 \cdot 10^{-9}}$	**0.0**	$\mathbf{2.775557561562891 \cdot 10^{-17}}$
10	$9.31 \cdot 10^{-10}$	0.0	$4.336808689942018 \cdot 10^{-19}$
15	$2.84 \cdot 10^{-14}$	0.0	$4.038967834731580 \cdot 10^{-28}$
20	$8.67 \cdot 10^{-19}$	0.0	$3.761581922631320 \cdot 10^{-37}$



Rysunek 1: Porównanie wartości $f(x)$ i $g(x)$ w zależności od k.

Interpretacja

- Wiarygodne są wyłącznie wyniki funkcji $g(x)$. We wzorze $f(x)$ od $k = 9$ wartość $x^2 \leqslant 2^{-54}$ staje się mniejsza niż próg precyzji, więc komputer zaokrągla $x^2 + 1$ do 1.0, dając błędne $f(x) = 0.0$.
- We wzorze $g(x)$ nie występuje odejmowanie bliskich sobie liczb, dlatego wyniki są poprawne dla wszystkich badanych k.

6 Zadanie 6

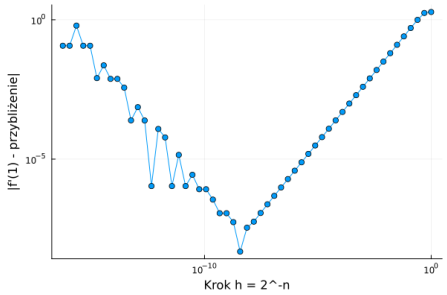
Obliczyłem przybliżenie pochodnej $f(x) = \sin x + \cos 3x$ w punkcie $x_0 = 1$ ilorazem różnicowym w przód:

$$\tilde{f}'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}, \qquad h = 2^{-n},\ n = 0, 1, \ldots, 54. \tag{2}$$

Wartość dokładna: $f'(1) = \cos(1) - 3\sin(3) \approx 0.116942281688538$.

Tabela 6: Przybliżenia pochodnej i błędy bezwzględne dla wybranych $h = 2^{-n}$.

n	$h = 2^{-n}$	$1 + h$	Przybliżenie $\tilde{f}'(1)$	Błąd bezwzględny
0	1.0	2.0000000000000000	2.017989	1.9010
5	$3.125 \cdot 10^{-2}$	1.0312500000000000	0.243443	$1.2650 \cdot 10^{-1}$
10	$9.766 \cdot 10^{-4}$	1.0009765625000000	0.120882	$3.9402 \cdot 10^{-3}$
20	$9.537 \cdot 10^{-7}$	1.0000009536743164	0.116946	$3.8473 \cdot 10^{-6}$
26	$\mathbf{1.490 \cdot 10^{-8}}$	**1.0000000149011612**	**0.11694234**	$\mathbf{5.70 \cdot 10^{-8}}$
27	$\mathbf{7.451 \cdot 10^{-9}}$	**1.0000000074505806**	**0.11694232**	$\mathbf{3.46 \cdot 10^{-8}}$
28	$\mathbf{3.725 \cdot 10^{-9}}$	**1.0000000037252903**	**0.11694229**	$\mathbf{4.80 \cdot 10^{-9}}$
35	$2.910 \cdot 10^{-11}$	1.0000000000291038	0.11693954	$2.7370 \cdot 10^{-6}$
50	$8.882 \cdot 10^{-16}$	1.0000000000000009	0.00000000	$1.1694 \cdot 10^{-1}$
52	$2.220 \cdot 10^{-16}$	1.0000000000000002	−0.5000000	$6.1694 \cdot 10^{-1}$
53	$\mathbf{1.110 \cdot 10^{-16}}$	**1.0000000000000000**	**0.00000000**	$\mathbf{1.1694 \cdot 10^{-1}}$



Rysunek 2: Błąd bezwzględny przybliżenia pochodnej w skali log-log.

Interpretacja

- Błąd całkowity stanowi sumę błędu metody $O(h)$ (dominującego dla dużych h) oraz błędu zaokrągleń $O(\epsilon/h)$ (dominującego dla małych h).
- Optymalny krok minimalizujący błąd wynosi $h_{\text{opt}} \approx \sqrt{\text{macheps}} = 2^{-26} \approx 1.49 \cdot 10^{-8}$; najmniejszy błąd bezwzględny w eksperymencie uzyskałem dla $n = 28$ ($4.80 \cdot 10^{-9}$).
- Dla $n \geqslant 53$ zachodzi $\text{fl}(1 + h) = 1.0$, przybliżenie pochodnej wynosi 0.0, a błąd bezwzględny stabilizuje się na poziomie dokładnej wartości $|f'(1)| \approx 0.1169$ (błąd względny 100%).